High-Level Design - British Auction in RFQ System

System Architecture, Anti-Sniping Engine & API Specifications

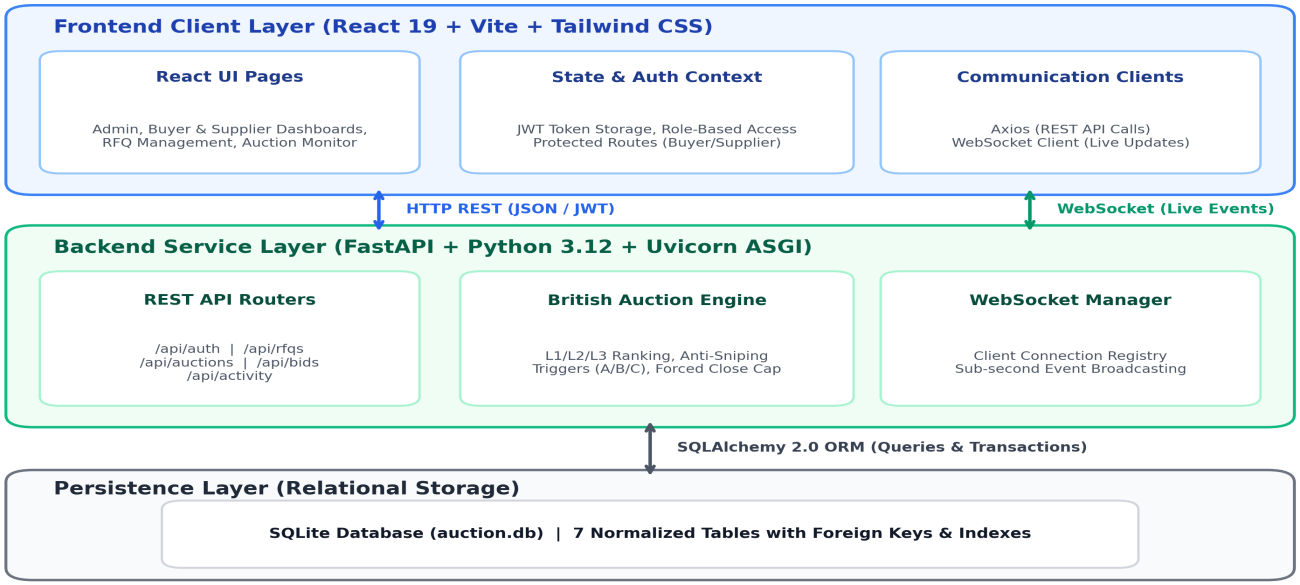
Project: British Auction RFQ Management System	**Architecture:** Decoupled 3-Tier Client-Server
Frontend: React 19, Vite, Tailwind CSS	**Backend:** Python FastAPI 0.115, Uvicorn ASGI
Real-Time: Native HTML5 WebSockets	**Database:** SQLite 3 with SQLAlchemy 2.0 ORM

1. Executive Overview & Problem Statement

In logistics procurement and commercial freight bidding, traditional fixed-deadline reverse auctions suffer from **bid sniping**: suppliers withhold their best rates until the final seconds to prevent counter-bids. This leads to artificial pressure, market distortion, and higher procurement costs for buyers.

The **British Auction in RFQ System** overcomes bid sniping through dynamic anti-sniping rules: whenever competitive bidding activity occurs within the final **X minutes** (trigger window), the auction closing deadline is dynamically extended by **Y minutes**. Simultaneously, an immutable **Forced Bid Close Time** hard ceiling guarantees definitive operational closure.

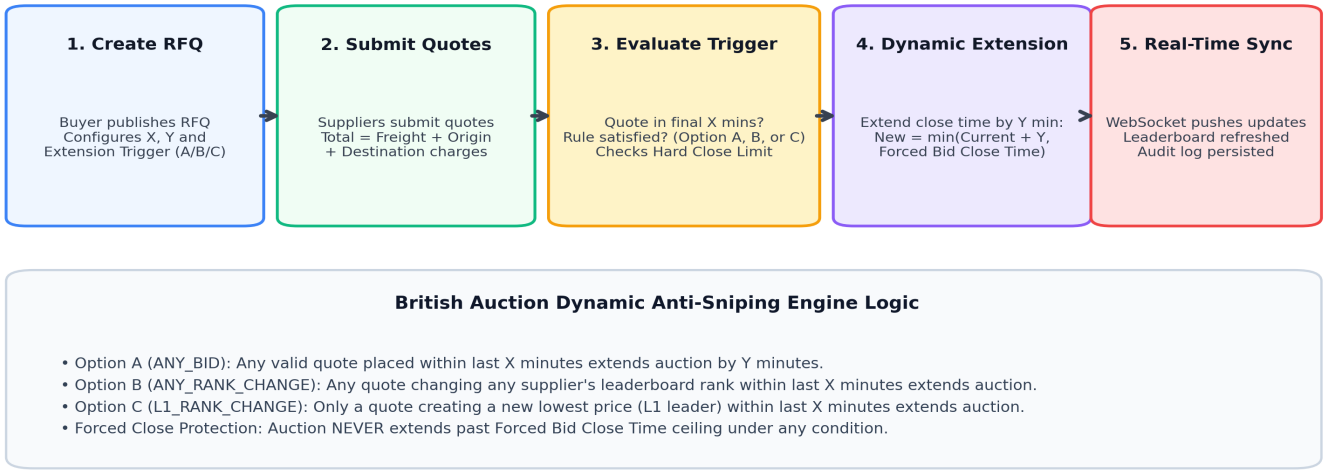
2. System Architecture



Frontend Client Layer	Backend API Layer	Persistence Layer
React 19 single-page application with Tailwind CSS. Role-based views for Admin, Buyer, and Supplier. Handles Axios REST API calls and persistent WebSocket streams.	FastAPI asynchronous service powered by Uvicorn. Implements JWT authentication, RBAC authorization, British Auction domain logic, and real-time WebSocket connection manager.	SQLite relational database managed via SQLAlchemy 2.0 ORM. Features 7 normalized tables, cascading constraints, unique indexes, and ACID transactional integrity.

3. End-to-End British Auction Workflow & Lifecycle

The operational lifecycle coordinates buyer RFQ publication, supplier quote submissions, anti-sniping trigger evaluation, and sub-second WebSocket broadcasting:



Step-by-Step Execution Sequence:

1. **RFQ Publication:** Buyer creates RFQ (origin, destination, cargo, schedule) and configures British Auction parameters: trigger window *X*, extension duration *Y*, extension trigger condition (Option A, B, or C), and forced close ceiling.
2. **Quote Submission:** Authorized suppliers submit detailed quotes (freight + origin + destination charges). The Auction Engine validates that quotes are strictly competitive (lower than the supplier's own previous best quote).
3. **Anti-Sniping Evaluation:** When a quote arrives within the final *X* minutes, the engine checks if the trigger condition is satisfied.
4. **Dynamic Time Extension:** If triggered, new closing time is calculated: *New Close = min(Current Close + Y, Forced Close)*. If Current Close + Y reaches or exceeds the forced ceiling, closing time clamps strictly to the ceiling.
5. **Real-Time Synchronization:** The WebSocket manager instantly pushes updated leaderboards, countdown clocks, and extension banners to all connected clients without polling.
6. **Auction Closure:** Once the countdown reaches zero or the forced close ceiling is hit, status transitions to CLOSED. No further bids are accepted.

4. Core British Auction Rules & Anti-Sniping Options

Rule Name	Implementation Specification	Engine Behavior & Constraint
Single Best Supplier Rule	Leaderboard aggregates exactly one entry per supplier.	Each supplier's lowest total quote is ranked (L1, L2, L3, ...). Earlier submission timestamp wins ties.
Option A: ANY_BID	Any valid quote placed within last *X* minutes.	Extends closing time by *Y* minutes, regardless of rank change. Maximizes competitive participation.
Option B: ANY_RANK_CHANGE	Any quote that shifts any position on leaderboard.	Extends if quote moves supplier up (e.g., L3 to L2 or L2 to L1). Improvements that don't change rank do not extend.
Option C: L1_RANK_CHANGE	Only a quote establishing a new overall lowest price (L1).	Extends only when market-leading rate is beaten. Default industry standard for procurement auctions.
Forced Close Limit	*New_Close = min(Close + Y, Forced_Close)*	Absolute hard ceiling. Bids attempted past forced close time are strictly rejected with HTTP 400.

5. API & WebSocket Specifications

The backend exposes RESTful endpoints and bi-directional WebSocket streams documented via OpenAPI/Swagger:

Method / Route	Access Role	Purpose & Description
`POST /api/auth/login`	Public	Authenticates user with email/password; returns JWT bearer token.
`GET /api/auth/me`	Authenticated	Resolves active session profile, role permissions, and supplier linkage.
`GET /api/rfqs`	Admin, Buyer	Retrieves list of RFQs with status filtering (All, Scheduled, Active, Closed).
`POST /api/rfqs`	Buyer	Creates a new RFQ with British Auction rules (X, Y, trigger type, forced close).
`GET /api/rfqs/{id}`	Authenticated	Fetches RFQ details, cargo specifications, and British Auction configuration.
`GET /api/auctions/{id}`	Authenticated	Fetches live auction countdown, extension logs, and ranked supplier leaderboard.
`POST /api/auctions/{id}/bids`	Supplier	Submits competitive quote; triggers ranking recalculation & anti-sniping extension.
`GET /api/auctions/{id}/bids`	Authenticated	Returns chronological bidding history and itemized quote charge breakdowns.
`GET /api/activity`	Authenticated	Returns immutable audit trail of system events, bids, and auction extensions.
`WS /ws/auctions/{id}`	Authenticated	Persistent WebSocket stream for sub-second quote and extension broadcast.

6. Security, Reliability & Quality Assurance

Security & Authentication: Passwords hashed using bcrypt. Stateless authentication via cryptographically signed JWT tokens with expiration. Role-Based Access Control (RBAC) enforced at middleware and router levels.

Concurrency & Race Condition Handling: Database transactions utilize scoped sessions with write locking to guarantee atomic quote evaluation, preventing simultaneous bids from corrupting leaderboard ranks or close times.

Automated Test Suite: Comprehensive test coverage (17/17 tests passing) validating all extension trigger types (A, B, C), tie-breaking, forced-close clamping, role authorization, and quote validation.